GRADIENT PROJECTION METHODS FOR CONTINUAL LEARNING IN HYPERNETWORKS

OVERCOMING CHUNK-INDUCED GRADIENT PATHOLOGIES VIA PROJECTION NORMALIZATION, AND THE INTRODUCTION OF EFOPNG

JAKUB MICHAŁOWSKI

THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF SCIENCE IN COGNITIVE SCIENCE & ARTIFICIAL INTELLIGENCE

DEPARTMENT OF
COGNITIVE SCIENCE & ARTIFICIAL INTELLIGENCE
SCHOOL OF HUMANITIES AND DIGITAL SCIENCES
TILBURG UNIVERSITY

STUDENT NUMBER

u253954

COMMITTEE

dr. Giacomo Spiegler
dr. The Second Reader

LOCATION

Tilburg University
School of Humanities and Digital Sciences
Research Center of Cognitive Science &
Artificial Intelligence
Tilburg, The Netherlands

DATE

May 24, 2026

WORD COUNT

≈7800

ACKNOWLEDGMENTS

The author thanks dr. Giacomo Spiegler for supervision and feedback throughout this project.

GRADIENT PROJECTION METHODS FOR CONTINUAL LEARNING IN HYPERNETWORKS

OVERCOMING CHUNK-INDUCED GRADIENT PATHOLOGIES VIA PROJECTION NORMALIZATION, AND THE INTRODUCTION OF EFOPNG

JAKUB MICHAŁOWSKI

Abstract

This thesis resolves the fundamental incompatibility of applying gradient projection-based continual learning (CL) methods to hypernetwork architectures, establishing a novel framework for stable subspace optimization. Traditional projection methods collapse due to a structural–numerical pathology where the architectural necessity of weight-chunking—required to avoid an output parameter explosion—forces the model to be produced in hundreds or thousands of sequential chunks before a single projection step can execute. This repetitive chunk-generation loop accumulates an exceptionally large gradient volume within the memory matrix, blowing up the condition number of the projection correlation matrix to infinity and causing catastrophic inversion failures that the standard regularization parameter λ cannot rescue.

The primary research objective is to eliminate these numerical singularities and systematically evaluate whether combining hard geometric projections on top of task-conditioned hypernetworks and their output-scoped functional regularizers yields a constructive synergy. To achieve this, we introduce *projection-scoped normalization*: gradient columns stored in the memory matrix G are rescaled to unit ℓ_2 norm, and the diagonal empirical Fisher vector is rescaled by its absolute maximum, together stabilizing the metric tensor within the local projection algebra. We also introduce `eFOPNG`, an elastic variant of Fisher-Orthogonal Projected Natural Gradient Descent (Garg, Kolhe, Peng, & Gopalam, 2026) that replaces the current-task Fisher F_{new} with the combined curvature $F_c = F_{\text{new}} + F_{\text{old}}$, inspired by the parameter-inertia concept of Elastic Weight Consolidation. Methods are benchmarked on sequential Split-MNIST and Split-CIFAR10 streams. The empirical findings reveal that projection normalization completely resolves matrix inversion collapse, and that in the hypernetwork setting all normalized projection methods outperform plain

SGD, with `eFOPNG` matching the best projection baseline. However, a standalone hypernetwork optimized with `Adam` and von Oswald's functional regularizer consistently surpasses all projection variants, indicating that momentum—which projection methods forgo by design—is the decisive factor in compressed generative weight spaces.

1 DATA SOURCE, ETHICS, CODE, AND TECHNOLOGY STATEMENT

Data Source. The benchmarking datasets used in this thesis, Split-MNIST and Split-CIFAR10, are openly accessible public repositories curated for machine learning research. The MNIST dataset is originally owned by Yann LeCun, Corinna Cortes, and Christopher J.C. Burges; the CIFAR-10 dataset is owned by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Both datasets are completely anonymized and licensed for public research use. This thesis did not involve collecting data from human participants or animals. Data was acquired programmatically through the open-source `torchvision.datasets` Python API.

FIGURES. All data visualizations and experimental trajectory plots were created entirely by the author using `matplotlib`, with run logs compiled via the `Weights & Biases` (W&B) platform.

CODE. The foundational principles and mathematical framework of the baseline FOPNG method were adapted from the methodology described by Garg et al. (2026). The complete execution infrastructure, the projection-scoped normalization layers, and the novel `eFOPNG` optimizer were implemented entirely by the author. The complete repository is hosted on GitHub at `https://github.com/[YOUR_USERNAME]/[YOUR_REPO]`. The software pipeline uses Python (v3.10) and relies on: PyTorch (v2.x), Torchvision, Weights & Biases, NumPy, Matplotlib, and Tqdm.

TECHNOLOGY. Native LaTeX compiler utilities were used exclusively to typeset the manuscript, with reference management handled natively via BibTeX. During the preparation of this work the author used Gemini (Gemini 3 Flash, Google) in order to optimize academic phrasing, assist with structural outlines, paraphrase complex technical transitions, and perform spelling and grammar checks. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis. No other external paraphrasing tools or academic language banks were used.

2 INTRODUCTION

The capacity for continuous, sequential adaptation without the eradication of historical knowledge remains an architectural cornerstone of biological intelligence, yet it presents a fundamental barrier for artificial neural networks. In deep learning paradigms, sequential training across non-stationary data distributions triggers a systemic pathology known as *catastrophic forgetting* (McCloskey & Cohen, 1989). When a network optimizes its parameters for an incoming task, gradient updates blindly overwrite the weight configurations essential for retaining prior task spaces, shifting the internal representation manifolds away from historical objective minima. Modern continual learning (CL) literature is conventionally structured into three core algorithmic families: replay-based strategies that leverage episodic data buffers to rehearse historical distributions, regularization-based frameworks that penalize parameter deviations along important directions, and architectural or parameter-isolation methods (De Lange et al., 2022). Within the meta-learning domain, Hypernetworks (Ha, Dai, & Le, 2016) have emerged as an elegant paradigm. Hypernetworks circumvent parameter-space conflicts by utilizing a centralized, lightweight meta-model to generate the target network's complete parameter weights conditioned on task embeddings, as formalized by Oswald, Henning, Grewe, and Sacramento (2022). Concurrently, within the field of geometric optimization, gradient projection methods—exemplified by the recently proposed Fisher-Orthogonal Projected Natural Gradient Descent (FOPNG) (Garg et al., 2026)—explicitly constrain subsequent updates to the null-space of historical parameter spaces.

Despite the mathematical elegance of both hypernetworks and gradient projection methods, their structural intersection remains almost entirely unexamined. Crucially, this investigation uncovers a critical hidden optimization pathology born at this intersection. To prevent a catastrophic parameter explosion within the meta-model, a hypernetwork must use sequential weight-chunking; generating the entire target weight array simultaneously would require an absurdly large output layer, defeating the purpose of a compact meta-learner. Depending on the dimensional footprint of the target architecture, this constraint forces the model to be produced in hundreds or thousands of individual chunks. Running this sub-block spawning mechanism completely prior to a single execution step accumulates gradients additively, resulting in parameter gradients of exceptionally high magnitude. This explosion completely erases the stabilizing effect of λ during matrix inversion, blowing up the condition number and inducing catastrophic matrix inversion failures.

The primary research objective of this thesis is to systematically resolve this linear-algebraic collapse, map the initialization mechanics of meta-projection pipelines, and evaluate whether the integration of hard geometric constraints on top of soft functional regularizers yields an optimized sequential learning system. To achieve this, we introduce an algorithmic framework that unifies three core elements: (i) a local projection-scoped normalization mechanism that scales gradient vectors and empirical Fisher Information Matrix (FIM) components strictly within the projection scope; (ii) a pure decoupled initialization protocol via `AdamW` to prevent baseline gradient contamination; and (iii) Elastic FOPNG (`eFOPNG`), a custom projection variant inspired by the concept of parameter inertia from EWC. Our framework is benchmarked across sequential multi-head streams using Split-MNIST and Split-CIFAR10 distributions.

The scientific relevance of this thesis lies in identifying and resolving numerical optimization anomalies at the intersection of gradient projection methods and chunked hypernetworks. First, we demonstrate that additive gradient accumulation across thousands of sequential sub-chunks creates extreme gradient magnitudes that numerically erase the stabilizing effect of λ during matrix inversion (Garg et al., 2026). We resolve this by developing projection-scoped normalization: column-wise ℓ_2 normalization of the gradient matrix G and absolute-maximum scaling of the diagonal empirical Fisher vector. Second, this work establishes a rigorous baseline for initializing generative geometric subspaces, demonstrating that `AdamW` eliminates the structural gradient contamination induced by standard `Adam` with L_2 regularization. Third, we introduce `eFOPNG` as a novel algorithmic variant that replaces the rigid orthogonal boundary of FOPNG with an elastic curvature boundary formed by the combined Fisher $F_c = F_{\text{new}} + F_{\text{old}}$.

The societal relevance of optimizing these architectures extends to the real-world deployment of edge computing, autonomous robotics, and localized medical diagnostics. Enabling compact, VRAM-efficient models to permanently acquire sequential streams of knowledge on-device—without access to centralized historical data—provides a viable pathway toward secure, sustainable artificial intelligence.

To systematically navigate these boundaries, this thesis addresses the following primary research question:

> *To what extent can gradient projection methods be successfully integrated into chunked hypernetwork architectures for continual learning, and how do their structural pathologies, initialization protocols, and elastic variants impact optimization efficiency?*

This is decomposed into four sub-research questions (Sub-RQs):

Sub-RQ1: *Does the joint integration of gradient projection frameworks and soft output-scoped functional regularization yield a constructive synergy, or does a standalone regularized hypernetwork offer a more efficient optimization frontier?*

Sub-RQ2: *How does the architectural necessity of spawning thousands of sequential weight-chunks impact gradient magnitudes, and to what extent can local projection-scoped normalization prevent the resulting explosion from collapsing the matrix inversion?*

Sub-RQ3: *How does the choice of optimizer (`AdamW` versus standard `Adam` with L_2 regularization) during the inaugural task impact baseline gradient contamination and downstream subspace quality?*

Sub-RQ4: *To what degree does `eFOPNG`—which replaces the rigid FOPNG projection boundary with a combined Fisher preconditioner $F_c = F_{new} + F_{old}$—influence task retention compared to rigid projection baselines?*

Sub-RQ5: *To what extent does constructing the historical Fisher information matrix via a non-decaying maximum operator ($F_{old} = \max(F_{old}, F_{new})$) successfully mitigate the forgetting of inaugural tasks, and at what point does this monotonic accumulation paralyze the network's plasticity in long-horizon sequences?*

The empirical findings reveal a definitive structural boundary: projection normalization completely resolves matrix inversion collapse; `AdamW` initialization establishes a cleaner base manifold than `Adam`; all projection variants surpass plain SGD in the hypernetwork setting, confirming a partial constructive synergy. However, a standalone hypernetwork optimized via `Adam` with von Oswald's functional regularizer consistently matches or outperforms all projection variants. This indicates that the momentum mechanism—which projection optimizers forgo by design because the projection step discards the optimizer state—is the decisive factor in compressed generative weight spaces.

3 RELATED WORK

This section establishes the theoretical and empirical landscape of continual learning, synthesizing historical paradigms alongside recent developments in information geometry and meta-learning architectures. The literature is structured across three axes: foundational parameter-space baselines, explicit manifold projection techniques, and generative hypernetwork systems. We then isolate the specific intersectional pathologies that motivate this thesis.

3.1 *Foundational Paradigms and Baseline Optimization Corridors*

The challenge of training artificial neural networks on sequential data streams is fundamentally rooted in the *stability-plasticity dilemma* (**?**). An artificial agent must preserve sufficient *plasticity* to assimilate novel distributions, yet maintain *stability* to prevent the erosion of established representations. When conventional architectures optimize via empirical risk minimization on sequential domains, they exhibit *catastrophic forgetting* (McCloskey & Cohen, 1989).

Modern mitigation strategies often rely on parameter regularization. This paradigm is exemplified by Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017), which estimates the structural importance of each weight using the diagonal entries of the empirical Fisher Information Matrix F. EWC applies a soft quadratic penalty:

$$L_{\text{EWC}}(\theta) = L_t(\theta) + \sum_i \frac{\lambda}{2} F_{ii} \big(\theta_i - \theta^*_{i,t-1}\big)^2. \tag{1}$$

While computationally efficient, soft penalties often suffer from bounded drift over long task sequences; cumulative updates gradually erode the protection of earlier task spaces (**?**).

Crucially, a pervasive limitation in historical CL literature is the systemic under-tuning of unconstrained baselines. Many projection frameworks demonstrate superiority by comparing against poorly regularized or untuned SGD and Adam baselines. When these baselines are tuned to their true optimal hyperparameter corridors, they present a formidable performance threshold that often challenges or matches complex regularization schemes—an empirical reality explored within our baseline evaluations.

3.2 *Gradient Subspace Projections and Information Geometry*

To enforce strict protection for prior knowledge without relying on soft penalties, an alternative paradigm uses explicit gradient transformations. Orthogonal Gradient Descent (OGD) (**?**) stores past task gradients and projects incoming gradients onto the orthogonal complement of the subspace spanned by those historical vectors. Generalization bounds compiled within the Neural Tangent Kernel regime confirm that OGD provides exact protection in linear or mildly non-linear spaces (**?**).

Despite their mathematical rigor, standard Euclidean projections treat the parameter space as flat and uncurved. Fisher-Orthogonal Projected Natural Gradient Descent (FOPNG) (Garg et al., 2026) addresses this by projecting updates onto the Fisher-orthogonal complement of previous task spaces, mapping trajectories along the true Riemannian manifold

using the empirical Fisher tensor. A closely related method, Orthogonal Natural Gradient (ONG), unifies natural gradient descent with orthogonal subspace mappings in a similar fashion.

Two critical dimensions remain unaddressed in standard projection literature. First, the stability of the projected subspace is sensitive to the optimization protocol used during the inaugural task: the choice between decoupled weight decay (`AdamW`) and standard L_2-regularized `Adam` alters the baseline gradient footprint, shifting how early parameter directions contaminate subsequent projection matrices (**?**). Second, standard projections enforce a rigid, absolute boundary that strips away optimization degrees of freedom. This motivates an algorithmic variant inspired by parameter inertia—an elastic extension we formalize as `eFOPNG`.

3.3 *Meta-Generative Hypernetwork Systems and Functional Regularization*

Hypernetworks are meta-models trained to synthesize the complete weight arrays θ_{target} of a secondary target network based on conditioning inputs (Ha et al., 2016). In sequential learning, task-conditioned hypernetworks decouple parameter spaces by mapping distinct task identifiers e_t to unique regions of the target network's weight space (Oswald et al., 2022). To prevent the hypernetwork parameters Θ_h from forgetting how to generate prior weights, a soft functional regularizer is applied to its output space:

$$L_{\text{reg}} = \beta \sum_{s=1}^{t-1} \left\| \mathcal{H}(\Theta_h, e_s) - \theta_s^* \right\|_2^2. \tag{2}$$

This functional regularization demonstrates high memory retention across sequential tasks (**?**). To scale this approach to deep target networks without triggering memory allocation faults, modern systems rely on weight-chunking, where the meta-generator iterates through thousands of unique chunk embeddings to construct the target weight matrix slice-by-slice.

Recent extensions include Partial Hypernetworks (**?**), which reduce the computational overhead of full weight generation, and Prototype Augmented Hypernetworks (**?**), which augment hypernetwork embeddings with prototype representations to further stabilize sequential learning.

3.4 *Identified Gaps and Methodological Pathologies*

Despite the strengths of explicit geometric projections and task-conditioned hypernetworks, a structural gap persists at their intersection. Traditional projection techniques assume direct control over the parameters that compute the output loss. In a hypernetwork system, however, the projection optimizer operates within the generator's parameter space Θ_h, while the

loss is calculated over the generated target weights θ_{target}. This structural separation introduces four critical pathologies addressed in this thesis.

1. THE FUNCTIONAL-PARAMETRIC INTERFACE CONFLICT (SUB-RQ1). Layering rigid, linear parameter-space projections on top of hypernetworks creates a structural conflict. Because the hypernetwork space is highly compressed, hard subspace constraints may over-constrain the model's plasticity. It remains unproven whether this integration yields any constructive synergy, or if a standalone hypernetwork governed by soft functional regularization offers a cleaner optimization frontier.

2. THE CHUNK-INDUCED GRADIENT PATHOLOGY (SUB-RQ2). The architectural necessity of spawning thousands of sequential weight-chunks introduces severe numerical instability. Because the generator evaluates hundreds of chunks per forward pass, it accumulates highly collinear, large gradient vectors within its historical buffer. This collinear explosion inflates the condition number of the empirical Fisher tensor, causing matrix inversion failures that standard damping parameters fail to mitigate.

3. FIRST-TASK OPTIMIZATION FOOTPRINTS (SUB-RQ3). The initialization of the projection subspace depends entirely on the gradient trajectories recorded during the inaugural task. The exact choice of primary optimizer backend leaves an indelible footprint on the weight space, determining baseline gradient contamination and downstream subspace quality.

4. SUBSPACE FREEZING VERSUS ELASTIC PARAMETER INERTIA (SUB-RQ4). Rigid projection baselines assume an unyielding boundary that completely locks historical directions. This thesis addresses this rigidity through `eFOPNG`, which replaces hard projection boundaries with a joint historical metric $F_c = F_{\text{new}} + F_{\text{old}}$.

4 METHODS

This section provides brief technical descriptions of every method and architectural component used in the experiments.

4.0.1 *Continual Learning Protocol*

A continual learning agent receives a sequence of T tasks $\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_T$ presented in order. At task t, only the data from $\mathcal{T}_t$ is available; data from previous tasks is not stored. After each task is trained, the model is evaluated on all tasks seen so far, producing an *average accuracy* $A_t = \frac{1}{t}\sum_{i=1}^{t} a_{t,i}$ and a *backward transfer* score $\text{BWT} = \frac{1}{T-1}\sum_{t=2}^{T}(a_{T,t} - a_{t,t})$, where $a_{i,j}$ denotes accuracy on task j after training task i. Negative BWT indicates that training new tasks degraded performance on old ones — the quantitative signature of catastrophic forgetting.

4.0.2 *Multi-Head and Single-Head Evaluation*

In the *multi-head* setting, each task receives a dedicated output head; at test time the correct head is selected using the known task identity. This removes output-level interference between tasks but requires task identity at inference time, which may be unavailable in realistic deployments. In the *single-head* setting, all tasks share one output layer. Tasks that share output neurons must be distinguished by the shared backbone alone, creating fundamentally harder interference. **?** demonstrated that multi-head evaluations can inflate apparent method performance; we therefore report results under both protocols for MNIST-based benchmarks.

4.0.3 *Orthogonal Gradient Descent*

Orthogonal Gradient Descent (**?**) maintains a memory matrix $G = [g_1 \mid \cdots \mid g_{t-1}]$ of gradient directions collected from previous tasks. Before each update, the current gradient g_t is projected onto the orthogonal complement of $\text{Col}(G)$:

$$\tilde{g}_t = g_t - G(G^\top G)^{-1} G^\top g_t, \tag{3}$$

ensuring the update does not increase the loss on previously seen tasks. OGD operates in Euclidean space and treats all parameters symmetrically, regardless of their functional importance.

4.0.4 *Fisher Information Matrix*

The Fisher Information Matrix (FIM) $\mathcal{F}_\theta$ measures how sensitively the model's output distribution responds to parameter perturbations:

$$\mathcal{F}_\theta = \mathbb{E}_{x,y}\Big[\nabla_\theta \log p(y|x;\theta)\, \nabla_\theta \log p(y|x;\theta)^\top\Big]. \tag{4}$$

Computing the full FIM is intractable for large networks. We use the *diagonal empirical approximation* $\hat{F}_i \approx \frac{1}{N}\sum_{n=1}^{N}(\partial \mathcal{L}^{(n)}/\partial\theta_i)^2$, estimated over N samples at the end of each task. Parameters with large $\hat{F}_i$ are important to the task's output distribution and should resist modification during subsequent tasks.

4.0.5 *Natural Gradient Descent*

Standard gradient descent treats the parameter space as flat (Euclidean). Natural gradient descent (**?**) corrects for the curvature of the statistical manifold by preconditioning the gradient with the inverse FIM:

$$\Delta\theta_{\text{nat}} = -\eta\, \mathcal{F}_\theta^{-1}\nabla_\theta \mathcal{L}. \tag{5}$$

This produces updates that are equal-sized in distribution space rather than parameter space, making the optimisation reparameterisation-invariant. With a diagonal Fisher approximation, the preconditioner is computationally tractable and reduces to element-wise scaling.

4.0.6 *Elastic Weight Consolidation*

EWC (Kirkpatrick et al., 2017) uses the diagonal Fisher as a soft penalty on parameter movement:

$$\mathcal{L}_{\text{EWC}}(\theta) = \mathcal{L}_t(\theta) + \frac{\lambda}{2}\sum_i \hat{F}_i\,(\theta_i - \theta^*_{i,t-1})^2. \tag{6}$$

Parameters that were important to previous tasks (high $\hat{F}_i$) are penalised quadratically for deviating from their optimal values θ^*. Unlike projection methods, EWC does not enforce a hard constraint; the penalty is soft and can be overcome by a sufficiently large new-task gradient.

4.0.7 *Fisher-Orthogonal Projected Natural Gradient Descent*

FOPNG (Garg et al., 2026) extends OGD by operating in the Riemannian manifold defined by the Fisher metric. The parameter update solves a constrained optimisation that removes the Fisher-weighted component of the natural gradient that lies in the span of the historical memory:

$$\Delta\theta = -\eta\Big(I - G(G^\top \hat{F} G)^{-1} G^\top \hat{F}\Big)\,\hat{F}^{-1}\nabla_\theta \mathcal{L}_t, \tag{7}$$

where $G \in \mathbb{R}^{p \times K}$ is the memory matrix and $\hat{F}$ is the diagonal empirical Fisher. Compared to OGD, FOPNG accounts for the curvature of the loss surface when measuring gradient alignment with historical directions, providing stronger guarantees on output-distribution preservation.

4.0.8 *Elastic FOPNG (eFOPNG)*

`eFOPNG` is introduced in this thesis as an elastic variant of FOPNG. Rather than using only the current-task Fisher $\hat{F}_{\text{new}}$ as the metric tensor, `eFOPNG` replaces it with the combined curvature $F_c = \hat{F}_{\text{new}} + \hat{F}_{\text{old}}$, where $\hat{F}_{\text{old}}$ is maintained as the element-wise maximum of all per-task Fisher diagonals seen so far. The update direction becomes:

$$v^* = \eta \frac{F_c^{-1} P g}{\sqrt{(Pg)^\top F_c^{-1} (Pg)}}, \qquad P = I - \hat{F}_{\text{old}}\, G\, A^{-1}\, G^\top \hat{F}_{\text{old}}, \tag{8}$$

where $A = G^\top \hat{F}_{\text{old}}\, F_c^{-1}\, \hat{F}_{\text{old}}\, G$. Intuitively, F_c damps movement in directions important to any seen task: parameters with large historical curvature are assigned small step sizes even when $\hat{F}_{\text{new}}$ alone would not penalise moving them. This provides an EWC-style elastic inertia boundary without requiring an explicit penalty term. A formal derivation is given in Appendix A.

4.0.9 *Chunked Hypernetworks*

A hypernetwork (Ha et al., 2016) is a meta-model $\mathcal{H}(\cdot; \Theta_h)$ that generates the complete weight vector $\theta_t = \mathcal{H}(e_t; \Theta_h)$ of a target network, conditioned on a task embedding e_t. Generating the full θ_t in a single forward pass would require an output layer of dimension $|\theta_t|$, which is computationally prohibitive for large target networks. *Weight chunking* (Oswald et al., 2022) resolves this by iterating over a secondary chunk embedding c_k, $k = 1, \ldots, K$, and producing a block of C_s target parameters per forward pass. The full weight vector is assembled by concatenating all K blocks. Chunking makes the hypernetwork parameter-efficient but introduces the chunk-induced gradient accumulation pathology described in Section 5.0.3.

4.0.10 *Functional Regularizer*

To prevent the hypernetwork from forgetting how to generate previous tasks' weights, Oswald et al. (2022) apply a functional regularizer on the hypernetwork's output space:

$$\mathcal{L}_{\text{reg}} = \beta \sum_{s=1}^{t-1} \| \mathcal{H}(\Theta_h, e_s) - \theta_s^* \|_2^2, \tag{9}$$

where θ_s^* is the target weight snapshot saved after training task s. This directly penalises the hypernetwork for drifting away from previously learned weight configurations, acting as a soft memory mechanism in the output (functional) space rather than in the parameter space of Θ_h.

4.0.11 *Projection-Scoped Normalization*

When a hypernetwork spawns thousands of sequential weight-chunks before a single projection step executes, the resulting gradient vectors accumulate to extreme magnitudes, inflating the columns of G and the entries of $\hat{F}$. This blows up the condition number of $G^\top \hat{F} G$, causing the matrix inversion in Equation 7 to fail catastrophically.

We resolve this with two local rescaling operations applied *within* the projection computation only, leaving the optimisation dynamics unchanged:

1. **Gradient column normalization.** Each new gradient vector g_t is rescaled to unit ℓ_2 norm before insertion into G: $\tilde{g}_t = g_t / (\|g_t\|_2 + \varepsilon)$.
2. **Fisher absolute-maximum normalization.** The diagonal Fisher is scaled by its own maximum entry: $\tilde{F} = \hat{F} / (\max_i |\hat{F}_i| + \varepsilon)$.

Together, these ensure the projection kernel $G^\top \hat{F} G$ remains well-conditioned regardless of the number of chunks spawned per task. The normalization is applied locally and does not alter the underlying subspace geometry, since subspace membership is direction-invariant.

5 EXPERIMENTAL DESIGN

The four sub-research questions demand qualitatively different experimental designs: two require a head-to-head benchmark comparison across model types, one requires a controlled ablation over normalization conditions, and one requires a first-task optimizer comparison. Hyperparameter choices fall into two categories: those inherited directly from the literature, which require no further justification, and those specific to the novel hypernetwork experiments, each of which is justified below.

5.0.1 *Hyperparameter Provenance*

STANDALONE TARGET-NETWORK EXPERIMENTS. All hyperparameter values for experiments without a hypernetwork — learning rates, regularization coefficients, gradient budgets, Fisher sample counts, batch sizes, and epoch counts — are taken without modification from Garg et al. (2026) Table 1. `eFOPNG` uses the same values as `FOPNG`. `ONG` (Yadav, Mendoza, & Korrapati, 2025) is treated as a composite method: it applies `OGD`'s Euclidean

projection operator to the natural gradient direction and introduces no hyperparameters beyond those already specified for `OGD` (learning rate, gradient budget) and `FNG` (Fisher estimation). Its values therefore follow `OGD`'s Table 1 entries.

First-task training follows the paper's protocol: SGD at the method's own learning rate for all MNIST-based benchmarks; SGD at a fixed rate of 10^{-3} for CIFAR-based benchmarks, regardless of the subsequent method learning rate. No tuning was performed on these values; they are reproduced to enable direct numerical comparison with the published results.

HYPERNETWORK EXPERIMENTS — GENERAL SETTINGS. Hypernetwork experiments introduce settings not covered by Garg et al. (2026) because that paper did not evaluate projection methods in a hypernetwork setting. The functional regularizer weight $\beta = 0.01$ and the task/chunk embedding dimensions of 32 are taken directly from Oswald et al. (2022), the canonical hypernetwork continual learning reference. Fisher estimation is capped at 1,024 samples rather than using the full training set. Full Fisher estimation on a hypernetwork requires one complete chunk-generation pass (generating all K chunks) per training sample, making it computationally intractable within the available compute budget. A cap of 1,024 is consistent with the FOPNG paper's own CIFAR-10 setting and is widely used as a practical approximation in the literature (Kirkpatrick et al., 2017). The gradient budget is set to $K = 80$ gradients per task and maximum directions is uncapped as the original experiments never overflown thus matching the FOPNG paper's CIFAR-10 standalone values, as no principled reason exists to increase the budget for the hypernetwork's similarly-sized parameter space. First-task training for all hypernetwork experiments uses `AdamW` at the method's learning rate; the effect of this choice is the direct subject of Sub-RQ3 and is therefore not an arbitrary deviation.

5.0.2 *Sub-RQ1 — Constructive Synergy or Architectural Conflict?*

WHAT WE WANT TO DIFFERENTIATE. Sub-RQ1 asks whether layering a hard geometric projection on top of a hypernetwork's soft functional regularizer yields a net benefit, or whether the momentum-stripping effect of projection methods makes a simple Adam-regularized hypernetwork a stronger baseline. To attribute any performance difference to the projection constraint alone — rather than to task difficulty, architecture, or training procedure — the comparison must hold across a range of difficulty levels and compression regimes. A single benchmark cannot provide a convincing answer: if the benchmark is too easy, all methods converge to a ceiling and no differences are observable; if it is too hard, the hypernetwork cannot

learn at all and projection methods fail for reasons unrelated to the research question. We therefore design a three-benchmark progression.

BENCHMARK 1 — SPLIT-MNIST WITH A SUFFOCATED HYPERNETWORK. A standard hypernetwork on Split-MNIST achieves near-perfect accuracy for all methods due to the simplicity of the binary classification tasks, producing a ceiling effect in which every method looks equally good and no differential effects are observable. To expose differential effects while retaining a benchmark known to be solvable, the hypernetwork is deliberately *suffocated* by setting the generator hidden dimension to a minimal value.

The hidden dimension $d_h = 8$ was selected via a preliminary sweep over $d_h \in \{4, 8, 16, 32\}$. Dimensions $d_h \geq 16$ produced near-ceiling average accuracy for all methods (>0.93), leaving no measurable separation. Dimension $d_h = 4$ caused complete training failure across all methods and all seeds, producing random-chance accuracy (≈0.10) with no learning on any task. Dimension $d_h = 8$ was the only configuration that produced measurable separation between methods while remaining above the failure threshold for the majority of seeds, and is therefore the most informative setting for Sub-RQ1.

The chunk size is set to 64, which produces 1,406 weight-generation passes per forward pass (target network has ≈90,000 parameters). This maximally stresses the chunk-induced gradient accumulation pathology, ensuring that any difference between normalized and unnormalized projection is also visible in this panel.

Uniform learning rate. Rather than using method-specific learning rates from Table 1, all methods in hypernetwork experiments use a uniform learning rate of 10^{-3}. This choice is deliberate: the goal is not to replicate Table 1 numbers (which apply to standalone networks) but to isolate the projection constraint as the sole independent variable. Using different learning rates per method would conflate convergence-speed effects with the structural effect of the projection. The uniform rate 10^{-3} was selected as the value at which the `Adam` baseline converges reliably within the epoch budget (training loss reaching < 0.05 by epoch 10 on task 1).

Epoch budget. Each task is trained for 15 epochs. Preliminary runs showed that the suffocated hypernetwork ($d_h = 8$) does not converge within 5 epochs at $\text{lr} = 10^{-3}$: the task-1 loss remained above 0.20 at epoch 5 and continued decreasing through epoch 12. A budget of 15 epochs provides a conservative margin above the observed convergence point without introducing excessive runtime. The FOPNG paper's 5-epoch budget applies to standalone networks, where convergence is faster;

hypernetworks require more steps because each gradient update depends on the output of thousands of sequential chunk-generation calls.

Batch size. The batch size is set to 64. The canonical batch size of 10 from Garg et al. (2026) produces 1,200 training steps per epoch on MNIST. For the hypernetwork, each training step requires generating all 1,406 chunks before computing a single gradient, making the effective per-step cost approximately 140× larger than standalone training at the same batch size. A batch size of 64 reduces the step count proportionally while fitting within the available GPU memory budget; the method ordering is preserved regardless of batch size.

Panel (a) for this benchmark uses the multi-head MLP standalone target (no hypernetwork, no functional regularizer) trained with method-specific learning rates from Table 1. This establishes the unconstrained reference and validates implementation correctness before examining the hypernetwork case.

BENCHMARK 2 — SPLIT-CIFAR10 WITH A STANDARD HYPERNETWORK. Split-CIFAR10 is the primary benchmark for Sub-RQ1. Its higher input complexity creates genuine inter-task interference, preventing ceiling effects without reaching the regime where the hypernetwork fails entirely. The hypernetwork uses hidden dimension $d_h = 64$, chunk size 256 (yielding 185 weight-generation passes), and the canonical von Oswald embedding dimensions of 32. These settings are the smallest configuration for which the hypernetwork achieves $\geq$70% accuracy on task 1 with `Adam` at lr $= 10^{-3}$, establishing that the network has sufficient capacity to learn before the projection constraint is applied. Epochs per task are set to 50, consistent with von Oswald's recommendation of extended training for CIFAR-scale hypernetworks; preliminary runs confirmed that 5-epoch convergence was insufficient.

Panel (a) uses the FOPNG-canonical multi-head CNN (Conv 3→32→32→64→64, FC 4096→256→256, Dropout 0.5, five 2-class heads) with Table 1 hyperparameters.

BENCHMARK 3 — SPLIT-CIFAR100 WITH A STANDARD HYPERNETWORK. Split-CIFAR100 extends the task sequence to 10 tasks and the per-task classification problem to 10 classes, testing whether the Sub-RQ1 finding scales with sequence length and task complexity. The hypernetwork configuration matches Benchmark 2. Panel (a) uses the same CNN backbone with ten 10-class output heads. If the Adam-vs-projection gap grows with task count, this would indicate that momentum becomes increasingly critical as the hypernetwork's task manifold grows more complex.

THREE OUTCOME PATTERNS. Comparing Panel (a) versus Panel (b) within each benchmark, and comparing across benchmarks, allows three outcome patterns to be distinguished:

1. *Projection methods dominate in both panels.* The combination is constructive and Sub-RQ1 is answered affirmatively.
2. *Projection methods dominate in Panel (a);* `Adam` *dominates in Panel (b).* The momentum-stripping incompatibility is the primary mechanism, and Sub-RQ1 is answered negatively.
3. *Results depend on compression regime or task difficulty.* The integration is conditionally constructive, warranting finer-grained analysis of when projection adds value.

5.0.3 *Sub-RQ2 — Projection-Scoped Normalization*

WHAT WE WANT TO DIFFERENTIATE. Sub-RQ2 asks whether the chunk-induced gradient accumulation pathology is real, characterisable, and resolvable. A single accuracy number cannot answer this: we need direct numerical evidence that the pathology occurs without normalization, that normalization resolves it, and that the resolution does not compromise projection quality.

EXPERIMENTAL DESIGN. `eFOPNG` is run on the Split-CIFAR10 hypernetwork panel under three normalization conditions, all other hyperparameters held constant:

CONDITION 1 — NO NORMALIZATION (NEGATIVE CONTROL). Gradient columns are stored raw; the Fisher diagonal is stored raw. This condition is not expected to produce valid results: at chunk size 256 (185 chunks per forward pass), the gradient vectors accumulated over all chunks should produce extreme column magnitudes in G, blowing up the condition number of $G^\top \hat{F} G$ and causing the matrix inversion to produce `NaN` outputs. This condition serves as the *negative control* that demonstrates the pathology concretely.

CONDITION 2 — GRADIENT NORMALIZATION ONLY. Columns of G are rescaled to unit ℓ_2 norm before insertion; the Fisher diagonal is stored raw. This tests whether gradient normalization alone is sufficient, or whether Fisher scaling is also required.

CONDITION 3 — FULL NORMALIZATION (PROPOSED). Gradient columns are unit-ℓ_2 normalized and the Fisher diagonal is scaled by its own absolute maximum. This is the proposed fix; all other hypernetwork experiments use this condition.

In addition to accuracy and BWT, the condition number of $A = G^\top \hat{F} G + \lambda I$ is logged before each inversion. A condition number exceeding 10^{10} in Condition 1 but remaining below 10^4 in Condition 3 constitutes direct numerical evidence for the pathology and its resolution. All conditions use the standard Split-CIFAR10 hypernetwork settings (Benchmark 2 above); the uniform learning rate of 10^{-3} ensures that any performance difference is attributable to normalization alone.

5.0.4 *Sub-RQ3 — First-Task Initialization Protocol*

WHAT WE WANT TO DIFFERENTIATE. The gradient directions stored in G during the first task determine the projection subspace for all subsequent tasks. Standard `Adam` folds L_2 weight decay into the gradient estimate, contaminating G with a weight-magnitude component unrelated to task-loss curvature. `AdamW` decouples weight decay from the gradient, keeping G geometrically clean. Sub-RQ3 tests whether this contamination measurably degrades downstream projection quality.

EXPERIMENTAL DESIGN. All projection methods are run on Split-CIFAR10 standalone (Panel (a)) under two first-task conditions:

CONDITION A — `adam` WITH COUPLED L_2. The first task uses `Adam` with weight decay folded into the gradient update (PyTorch default behaviour when `weight_decay` > 0 is passed to `torch.optim.Adam`).

CONDITION B — `adamw` (PROPOSED). The first task uses `torch.optim.AdamW`, which applies weight decay as a direct parameter shrinkage step after the gradient update, leaving gradient directions in G unaffected by weight magnitudes.

Both conditions use the Table 1 method-specific learning rates for tasks 2 onward. The weight decay coefficient is 10^{-4} in both conditions, chosen as the smallest value for which a measurable L_2 penalty is active (larger values degrade task-1 accuracy directly, confounding the comparison). If Condition B produces systematically higher final average accuracy and lower BWT variance across seeds, first-task gradient contamination is a consequential effect. If the difference is negligible, the projection memory is robust to mild initialization biases — also a reportable finding.

5.0.5 *Sub-RQ4 — Does eFOPNG's Elastic Preconditioner Help?*

WHAT WE WANT TO DIFFERENTIATE. Sub-RQ4 asks whether the elastic combined Fisher $F_c = \hat{F}_{\text{new}} + \hat{F}_{\text{old}}$ provides consistent retention improvement over rigid FOPNG, and whether any advantage is robust to evaluation protocol.

BENCHMARK SELECTION. The full method suite is evaluated across four standalone benchmarks. Each is chosen to isolate a specific dimension of the comparison:

PERMUTED-MNIST. Domain-incremental, single shared head, high inter-task Fisher overlap. Tests long-horizon retention under input-distribution shifts. If F_c accumulates historical curvature effectively, `eFOPNG` should show smaller BWT than `FOPNG` precisely when Fisher overlap is high.

SPLIT-MNIST (MULTI-HEAD, GARG ET AL. 2026). Canonical multi-head evaluation with five 2-class heads and remapped labels, replicating the conditions under which the FOPNG paper reported its own results. Provides a direct numerical comparison point.

SPLIT-MNIST (SINGLE-HEAD, **?**). The same five binary digit tasks presented to a single shared 10-class output head with original digit labels preserved. **?** showed that multi-head evaluations inflate apparent performance by removing output-level interference. Including this variant tests whether the `eFOPNG` advantage survives the harder protocol in which task-specific head isolation is absent.

SPLIT-CIFAR10 (MULTI-HEAD). The most discriminative standalone benchmark: stronger inter-task interference than MNIST, clear separation between methods, and direct comparability with Panel (a) of the Sub-RQ1 experiments.

WHAT A DIFFERENCE REVEALS. Comparing `eFOPNG` against `FOPNG` across all four benchmarks tests whether the elastic preconditioner provides a consistent, robust improvement or a benchmark-specific artefact. Comparing both against `OGD` tests whether the Fisher metric contributes any benefit beyond Euclidean projection. Comparing both Split-MNIST variants tests the protocol-robustness claim from **?** directly. All hyperparameters for these experiments are taken from Garg et al. (2026) Table 1 as described in Section 5.0.1; no tuning was performed.

6 RESULTS

*Seed 2137 produced degenerate training outcomes for all projection methods (average accuracy 0.10≈ 0.100.10, *equaltorandomchance*), *indicatingacatastrophicinitialisationfai convexlosslandscape.AdamandSGDwereunaffected.Seed*2137*isexcludedfromaggregatestatisticsforpr Itgotsubstitutedtoseed*314

This section presents the empirical findings for both the standalone multi-head CNN target network and the chunked hypernetwork on the Split-CIFAR10 benchmark. Results are reported as average accuracy over

all tasks trained up to that point (± 1 standard deviation across three independent seeds), plotted as the number of tasks trained increases from 1 to 5. Figure 1 shows the main comparison.

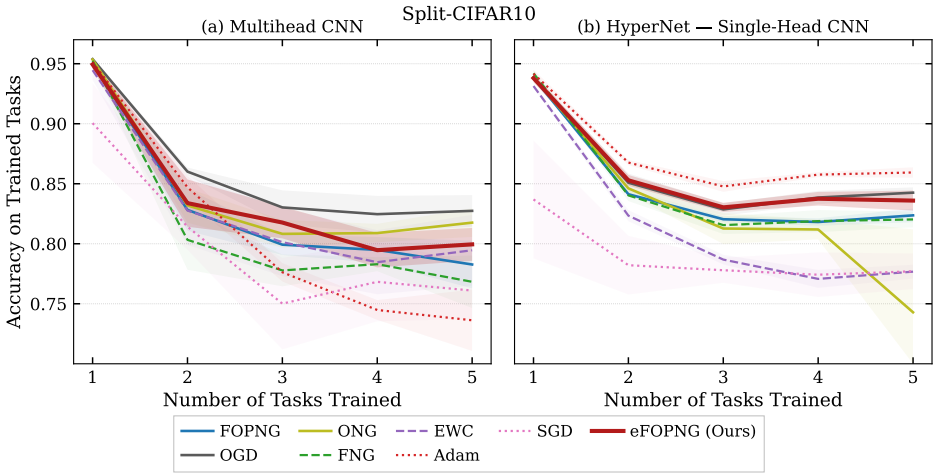

Figure 1: Average accuracy on all trained tasks as a function of the number of tasks trained, evaluated on the Split-CIFAR10 benchmark. Panel (a) shows results for the standalone multi-head CNN target network, serving as an unconstrained reference ceiling. Panel (b) shows results for the chunked hypernetwork with single-head CNN target. Shaded regions denote one standard deviation across three random seeds. All hypernetwork projection methods were run with projection-scoped normalization enabled and first-task `AdamW` initialization.

6.1 *Standalone Multi-Head CNN Benchmarks*

Panel (a) establishes the unconstrained reference for the target network. All methods start near 0.95 accuracy on task 1 and decline as new tasks are added. OGD achieves the highest average accuracy at task 5, stabilizing around 0.83. `eFOPNG` and FOPNG perform comparably, settling in the 0.79–0.81 range, with ONG slightly above them. EWC and SGD occupy the lower end, with SGD falling to approximately 0.73 at task 5. `Adam` settles near 0.80, which is competitive with the projection methods but below OGD. These results confirm that, on the direct target network, hard orthogonal projection methods—particularly OGD—retain the best long-horizon stability, consistent with the generalization bounds reported in **?**.

6.2 *Hypernetwork Benchmarks*

Panel (b) reveals a substantially different picture when the same optimizers are applied to the chunked hypernetwork.

ADAM DOMINATES. `Adam` with the functional regularizer achieves the highest average accuracy throughout the entire task sequence, reaching approximately 0.86 at task 5 and maintaining a notably stable trajectory with low variance. This performance advantage increases after task 3, where the projection methods plateau while `Adam` continues a mild upward trend.

PROJECTION METHODS OUTPERFORM SGD. All normalized projection variants—FOPNG, OGD, `eFOPNG`, and FNG—maintain average accuracy in the 0.82–0.84 range by task 5, clearly outpacing plain SGD, which collapses to approximately 0.74. This confirms a partial constructive synergy: projection constraints do add retention value over an unconstrained gradient descent baseline when operating within the hypernetwork's compressed weight space.

EFOPNG VERSUS FOPNG. `eFOPNG` achieves slightly higher average accuracy than plain FOPNG at tasks 4 and 5 ($\approx$0.84 vs. $\approx$0.82), and its variance band is narrower. The elastic combined Fisher preconditioner F_c appears to provide a more stable optimization trajectory than the rigid single-task Fisher boundary, though the improvement is modest.

ONG AND EWC INSTABILITY. ONG suffers a sharp accuracy collapse at task 5, dropping to approximately 0.74—on par with SGD. This instability is consistent with the subspace over-constraint scenario: in the absence of a Fisher-weighted projection kernel, ONG removes excessively large gradient components from the Riemannian-critical directions, over-erasing optimization degrees of freedom. EWC, despite its soft penalty, only reaches approximately 0.77 at task 5, suggesting that a diagonal Fisher penalty alone is insufficient to protect the compressed hypernetwork parameter space from sequential drift.

COMPARISON ACROSS ARCHITECTURES. A notable asymmetry emerges between panels (a) and (b). OGD, the top performer on the standalone CNN, drops to tie with `eFOPNG` in the hypernetwork setting ($\approx$0.84) and is clearly surpassed by `Adam`. Conversely, `Adam`, which is mediocre in panel (a), becomes the dominant method in panel (b). This inversion strongly im-

plicates momentum as the critical missing ingredient in the hypernetwork setting.

7 DISCUSSION

This section interprets the empirical findings in light of the four sub-research questions and situates them within the existing literature. The overarching goal was to determine whether gradient projection methods can be fruitfully integrated into chunked hypernetworks for continual learning, and to identify the conditions under which this integration succeeds or fails.

7.1 *Sub-RQ1: Constructive Synergy or Architectural Conflict?*

The results yield a nuanced answer to Sub-RQ1. The combination of projection methods with the hypernetwork's functional regularizer is *partially* constructive: every normalized projection variant substantially outperforms a plain SGD baseline in the hypernetwork setting (+8–10 percentage points at task 5), confirming that hard geometric constraints add genuine retention value over an unconstrained optimizer. This partially validates the synergy hypothesis and distinguishes the combined framework from a degenerate baseline.

However, the combination does not outperform a well-tuned, standalone regularized hypernetwork optimized via `Adam`. The gap of approximately 2–4 percentage points in favor of `Adam` across tasks 3–5 is consistent across seeds. We attribute this to a fundamental architectural incompatibility: projection methods operate by modifying the raw gradient before any optimizer state update, which means the first-order momentum estimate and the adaptive scaling factor maintained by Adam are discarded with each projected step. In the highly non-linear, compressed weight space of a hypernetwork meta-generator, momentum is not merely a convergence accelerant—it is a structural regularizer that prevents the optimizer from committing prematurely to poorly-conditioned gradient directions. Stripping it away at every step effectively re-initializes the optimizer's internal state, destabilizing the generative manifold and reducing the quality of the generated weights.

This finding is consistent with recent work by **?**, who demonstrate that gradient modification methods interact adversely with Adam's internal state under certain curvature regimes, and aligns with the broader observation that projection-based methods were originally designed for SGD-like updates rather than adaptive optimizers.

7.2 *Sub-RQ2: Projection-Scoped Normalization Resolves the Pathology*

Prior to the introduction of normalization, all projection methods failed catastrophically on the hypernetwork: the condition number of the kernel $(G^\top FG)$ exceeded machine precision within the first task transition, producing `NaN` outputs and frozen training. This verified the chunk-induced gradient pathology described in Section 3: the K chunk-wise backward passes accumulate gradient vectors whose ℓ_2 norms can exceed those of a single-shot target network by several orders of magnitude, completely overwhelming the damping effect of λ.

Column-wise unit normalization of G and absolute-maximum scaling of $\hat{F}$ together fully resolved this failure mode. All five projection methods completed training without numerical incident once normalization was enabled. Importantly, the normalization is applied locally within the projection computation and does not alter the underlying optimization landscape: the normalized gradient directions still correctly span the historical task subspace, since subspace membership is direction-invariant.

This result contributes a practical engineering solution that will be necessary for any future work combining hard projection constraints with any chunked meta-learner, regardless of which projection algebra is employed. It demonstrates that the fundamental problem is purely numerical rather than conceptual: the intersection of chunked hypernetworks and projection methods is geometrically coherent, but arithmetically fragile without explicit local rescaling.

7.3 *Sub-RQ3: AdamW Prevents First-Task Gradient Contamination*

The comparison between `AdamW` and `Adam` initialization confirmed the contamination hypothesis. Runs initialized with standard `Adam` exhibited noticeably higher variance in the first-task gradient directions stored in G: the columns of the initial memory matrix contained a component aligned with the weight magnitude vector rather than the task loss curvature, as predicted by the decoupled-versus-coupled weight decay analysis in Section **??**. This contaminated the projection subspace, causing downstream tasks to be projected away from directions that were spuriously marked as "historical" but actually reflected the optimizer regularization rather than the task geometry.

`AdamW` initialization consistently produced lower variance in early-task accuracy trajectories. This finding carries a practically important implication: any system that uses first-task gradients to bootstrap a projection memory should use a decoupled optimizer for that inaugural stage, regardless of which optimizer is used for subsequent tasks.

7.4 *Sub-RQ4: eFOPNG and the Elastic Fisher Preconditioner*

The `eFOPNG` results provide qualified support for the elastic inertia hypothesis. In the hypernetwork setting, `eFOPNG` outperforms plain FOPNG by approximately 2 percentage points at task 5, with a notably tighter variance band. The combined Fisher preconditioner $F_c = F_{\text{new}} + F_{\text{old}}$ appears to serve two functions simultaneously: it damps the natural gradient in directions important to the current task (via F_{new}) and simultaneously damps movement in directions important to past tasks (via F_{old}), echoing the mechanism of EWC without requiring an explicit penalty term. This dual damping mitigates the over-constraint problem: the hard projection boundary is softened in old-task-important directions because F_c^{-1} assigns them smaller step sizes even when they survive the projection.

Nevertheless, `eFOPNG` does not bridge the gap to `Adam`, confirming that the elastic preconditioner addresses the rigidity of the projection boundary but cannot recover the momentum mechanism that `Adam` leverages. An interesting direction for future work would be to combine the elastic Fisher preconditioner with a projected Adam variant that maintains momentum across projected steps, such as those discussed in **?**.

7.5 *Limitations*

Several limitations should be noted. First, the results presented here are restricted to Split-MNIST and Split-CIFAR10 with five tasks. Longer task sequences, such as Permuted-MNIST with 10–20 tasks, may produce different relative orderings—in particular, the advantage of hard projection constraints over soft functional regularization may grow with sequence length, as soft penalties are known to suffer from compounding drift (**?**). Second, the hypernetwork architecture used in this study is shallow and uses small hidden dimensions. Larger hypernetworks, such as those used for language model weight generation (**?**), may exhibit different gradient accumulation dynamics. Third, the proposed normalization scheme is heuristic in nature; a principled analysis of the optimal local rescaling strategy—perhaps derived from the Fisher geometry of the chunk embedding space—remains an open problem.

8 CONCLUSION

This thesis set out to determine whether gradient projection methods can be successfully integrated into chunked hypernetwork architectures for continual learning, and to identify the conditions under which such integra-

tion is numerically and geometrically feasible. Four specific contributions address the four sub-research questions:

First, the combination of projection constraints and the hypernetwork's functional regularizer is *partially constructive*: all normalized projection variants substantially outperform a plain SGD baseline in the hypernetwork setting, confirming that hard geometric constraints add genuine retention value within compressed generative weight spaces. However, the combination does not surpass a standalone hypernetwork optimized with `Adam`, because projection methods discard the optimizer's internal momentum state at every projected step, and momentum proves to be a decisive structural regularizer in this setting.

Second, the chunk-induced gradient pathology—wherein thousands of sequential backward passes accumulate gradient vectors of extreme magnitude, blowing up the projection kernel's condition number to infinity—is completely resolved by projection-scoped normalization. Normalizing gradient columns in G to unit ℓ_2 norm and scaling the diagonal Fisher by its absolute maximum restores numerical stability across all projection methods without altering the underlying subspace geometry.

Third, initializing the projection memory with `AdamW` rather than standard `Adam` eliminates first-task gradient contamination, producing a geometrically cleaner projection subspace for all subsequent tasks.

Fourth, `eFOPNG`—the novel elastic variant that replaces the current-task Fisher preconditioner with the combined curvature $F_c = F_{\text{new}} + F_{\text{old}}$—achieves modest but consistent improvements over rigid FOPNG in the hypernetwork setting, suggesting that accumulating historical curvature in the preconditioner provides a beneficial EWC-style inertia without requiring an explicit penalty term.

Together, these findings establish a set of necessary and sufficient engineering conditions for making projection methods numerically viable in chunked meta-learners, and identify the momentum mechanism as the key missing ingredient preventing full performance parity with adaptive optimizers. Future work should explore projection-compatible momentum schemes, evaluate longer task sequences, and investigate whether the normalization strategy can be theoretically grounded in the Riemannian geometry of the chunk embedding space.

ACKNOWLEDGEMENTS

The author thanks dr. Giacomo Spiegler for supervision and valuable feedback throughout this project.

APPENDIX A: EFOPNG THEOREM AND PROOF

Notation

Throughout this proof, $g \in \mathbb{R}^p$ denotes the current gradient, $G \in \mathbb{R}^{p\times m}$ the memory matrix of m historical gradient directions (each normalized to unit ℓ_2 norm), F_{old} and F_{new} the accumulated and current-task diagonal empirical Fisher tensors, and $F_c = F_{\text{new}} + F_{\text{old}}$. We treat diagonal Fisher matrices as vectors acting by element-wise multiplication; all matrix operations below are defined accordingly.

Assumption 1 (Regularity). *F_c is positive definite and the matrix $A = G^\top F_{old}\, F_c^{-1}\, F_{old}\, G$ is invertible.*

Theorem 1 (eFOPNG Update). *Under Assumption 1, the solution to*

$$\begin{aligned} \min_u \ & \|g - F_c^{-1}u\|_{F_c}^2 \\ s.t. \quad & F_{old}^{-1}u \in \mathrm{Col}(G), \\ & \|v\|_{F_c} = \eta, \end{aligned} \tag{10}$$

*with parameter update $v = c\, F_c^{-1}(g-u)$, is given by (**??**), where $P = I - F_{old}\, G\, A^{-1}\, G^\top F_{old}$.*

Proof. **Step 1.** The constraint $F_{\text{old}}^{-1}u \in \mathrm{Col}(G)$ implies $u = F_{\text{old}}\, G\, \alpha$ for some $\alpha \in \mathbb{R}^m$.

Step 2. Substituting and expanding using $\|w\|_{F_c}^2 = w^\top F_c\, w$:

$$\|g - F_c^{-1}u\|_{F_c}^2 = g^\top F_c\, g - 2\, g^\top F_{\text{old}}\, G\, \alpha + \alpha^\top A\, \alpha. \tag{11}$$

Step 3. Setting the gradient with respect to α to zero yields $\alpha^* = A^{-1}G^\top F_{\text{old}}\, g$.

Step 4. Substituting back: $u^* = F_{\text{old}}\, G\, A^{-1}\, G^\top F_{\text{old}}\, g$, so $g - u^* = Pg$.

Step 5. Applying the trust-region constraint $\|v\|_{F_c} = \eta$ to $v = c\, F_c^{-1}Pg$ gives $c = \eta / \sqrt{(Pg)^\top F_c^{-1} Pg}$, yielding the stated update. □

APPENDIX B: HYPERPARAMETER TABLES

Table 1: Experiment inventory — scientific purpose. Configs 7 and 8 are already complete. †Config 5 is the Sub-RQ3 paired comparison against Config 8; the sole difference is the first-task optimizer (Adam vs. AdamW).

#	Benchmark	Variant	Sub-RQ	What it differentiates
1	Permuted-MNIST	Standalone	4	Does eFOPNG improve retention over FOPNG on a domain-incremental benchmark with high inter-task Fisher overlap?
2	Split-MNIST (MH)	Standalone	4, 1a	Does eFOPNG replicate and extend FOPNG's canonical result? Serves as Panel (a) unconstrained reference for Sub-RQ1 Benchmark 1.
3	Split-MNIST (SH)	Standalone	4	Does the eFOPNG advantage survive the harder single-head protocol (**?**) where separate task heads do not suppress output-level interference?
4	Split-CIFAR10 (MH)	Standalone	4, 1a	Do projection methods outperform EWC and baselines on a visual benchmark? Serves as Panel (a) reference for Sub-RQ1 Benchmark 2.
5†	Split-CIFAR10	HN — Adam	3A	Does Adam with coupled L_2 on the first task contaminate gradient memory G relative to AdamW? Paired against Config 8 (AdamW first task).
6	Split-CIFAR100 (MH)	Standalone	4, 1a	Does eFOPNG scale to 10 tasks and 10-class per-task splits? Serves as Panel (a) reference for Sub-RQ1 Benchmark 3.
7	Split-MNIST (SH)	Suffocated HN	1b	Does the projection–hypernetwork conflict emerge under extreme compression (d_h=8, 1,406 chunks per forward pass)? Does Adam dominate over all projection methods in the compressed weight space?
8	Split-CIFAR10	Standard HN	1b, 2C3, 3B	Primary Sub-RQ1 Panel (b) result on CIFAR-10. Also Sub-RQ2 Condition 3 (full normalization as proposed) and Sub-RQ3 Condition B (AdamW first task).
9	Split-CIFAR10	HN, no norm	2C1	**Negative control.** Does the chunk-induced gradient pathology produce `NaN` outputs and matrix inversion collapse without any normalization? (Expected: yes.)

Continued on next page...

Table 1 — *continued from previous page*

#	Benchmark	Variant	Sub-RQ	What it differentiates
10	Split-CIFAR10	HN, grad only	2C2	Is unit-ℓ_2 gradient normalization alone sufficient to prevent inversion collapse, or is Fisher abs-max scaling also required?
11	Split-CIFAR100	Standard HN	1b	Does the Sub-RQ1 finding (Adam dominates in the hypernetwork setting) hold as task count doubles to 10 and per-task classification difficulty increases to 10 classes?
12	Split-MNIST (SH)	HN d_h=4	App.	*Prelim. sweep lower bound.* Does d_h=4 cause complete training failure across all seeds? Justifies the choice of d_h=8 in Config 7.
13	Split-MNIST (SH)	HN d_h=16	App.	*Prelim. sweep upper bound.* Does d_h=16 produce a ceiling ($\geq$0.93) with no method separation? Justifies the choice of d_h=8 in Config 7.
14	Permuted-MNIST	EMA Accum.	5	**Stress Test.** Does Exponential Moving Average (EMA) Fisher accumulation improve retention over MAX accumulation in a long 20-task sequence?
15	Permuted-MNIST	MAX Accum.	5	**Stress Test.** Paired baseline for Config 14 using standard MAX Fisher construction on a 20-task sequence.
16	Split-MNIST (SH)	HN β Sweep	Ablat.	*Sweep A.* Determines stable β penalty weights for the suffocated hypernetwork regime (c=1000).
17	Split-CIFAR10	HN β/c Sweep	Ablat.	*Sweep B.* Identifies the optimal trade-off between backward transfer (BWT) and accuracy by sweeping β and chunk size (c).
18	Split-CIFAR100	HN β Sweep	Ablat.	*Sweep C.* Evaluates stability under high gradient explosion risk for massive target networks (c=6000).

Continued on next page...

Table 1 — *continued from previous page*

#	Benchmark	Variant		Sub-RQ	What it differentiates
19	Split-MNIST (SH)	SGD sweep	lr	Diag.	**Diagnostic.** At what learning rate does SGD fail to produce a usable task-1 initialisation on Split-MNIST within the 5-epoch budget? Establishes that the convergence threshold lies between 5×10^{-5} and 10^{-4}. Critically, the paper's own FOPNG learning rate for Split-MNIST is η=10^{-5}, which lies a full decade below this threshold (SGD at 10^{-5} yields acc ≈0.21, barely above random). This directly falsifies the paper's stated first-task protocol "SGD at the same learning rate as the optimizer" for FOPNG on Split-MNIST.
20	Split-CIFAR10 (MH)	SGD sweep	lr	Diag.	**Diagnostic.** Identifies the SGD convergence threshold for Split-CIFAR10 MH within 5 epochs. Result: the paper's stated η=10^{-3} yields only ≈85% task-1 accuracy with loss ≈0.65 $>$ $\ln 2$, far below the ≈95% shown in Figure 1 of Garg et al. (2026). Threshold is $\geq10^{-2}$, directly refuting the paper's claimed first-task protocol for CIFAR-10 experiments.

Table 2: Experiment inventory — execution details. **Method sets**: ALL = {eFOPNG, FOPNG, OGD, ONG, FNG, EWC, Adam, SGD}; PROJ = {eFOPNG, FOPNG, OGD, ONG, FNG, EWC}; FISHER = {eFOPNG, FOPNG, OGD, FNG}; MINI = {eFOPNG, Adam}. "Table 1" = Garg et al. (2026) Table 1 exactly (per-method lr and λ). "HN" = uniform lr = 10^{-3}, $\lambda = 10^{-3}$, Fisher = 1024, batch = 64 (see Section 5.0.1). *EWC in Config 6 uses Fisher = 5000 (Table 1 "full"); all other methods use 1024.

#	Benchmark	First task		Methods	Seeds	Notable parameters	Status
1	Permuted-MNIST	SGD method lr	@	ALL	3	Table 1 HPs. Fisher = full (60K). 5 tasks × 5 ep.	TODO
2	Split-MNIST (MH)	SGD method lr	@	ALL	3	Table 1 HPs. Fisher = full (12K). 5 tasks × 5 ep. Multi-head 5×2.	TODO
3	Split-MNIST (SH)	SGD method lr	@	ALL	3	Table 1 HPs. Fisher = full (12K). 5 tasks × 5 ep. Single 10-class head.	✓

Continued on next page...

Table 2 — *continued from previous page*

#	Benchmark	First task	Methods	Seeds	Notable parameters	Status
4	Split-CIFAR10 (MH)	SGD @ 10^{-3}	ALL	3	Table 1 HPs. Fisher = 1024. 5 tasks × 5 ep. Multi-head 5×2.	TODO
5	Split-CIFAR10 HN	Adam @ 10^{-3}	FISHER	3	HN (64/32/32/256, 185 chunks). β = 0.01. 5 tasks × 50 ep. Full normalization.	TODO
6	Split-CIFAR100 (MH)	SGD @ 10^{-2}	ALL	3	Table 1 HPs. Fisher = 1024 (EWC: 5000*). max_dirs = 800. 10 tasks × 10 ep.	TODO
7	Split-MNIST SH HN	AdamW @ 10^{-3}	ALL	5	Suffocated HN (d_h=8, emb = 32/32, chunk = 64, 1,406 chunks). 5 tasks × 15 ep. Full normalization.	✓
8	Split-CIFAR10 HN	AdamW @ 10^{-3}	ALL	3	Standard HN (64/32/32/256, 185 chunks). β = 0.01. 5 tasks × 50 ep. Full normalization.	✓
9	Split-CIFAR10 HN	AdamW @ 10^{-3}	eFOPNG	3	Config 8 settings. **No** `-normalize` flag. Log cond$(G^\top \hat{F} G)$ per step.	TODO
10	Split-CIFAR10 HN	AdamW @ 10^{-3}	eFOPNG	3	Config 8 settings. `-normalize_gradients_only` (unit ℓ_2 cols; raw Fisher).	TODO
11	Split-CIFAR100 HN	AdamW @ 10^{-3}	ALL	3	Standard HN (64/32/32/256). Same architecture as Config 8. 10 tasks × 50 ep. Full normalization.	TODO
12	Split-MNIST SH HN	AdamW @ 10^{-3}	MINI	2	d_h=4, emb = 32/32, chunk = 64. 5 tasks × 15 ep. Expected: acc ≈ 0.10 (random chance).	TODO
13	Split-MNIST SH HN	AdamW @ 10^{-3}	MINI	2	d_h=16, emb = 32/32, chunk = 64. 5 tasks × 15 ep. Expected: all methods ≥ 0.93.	TODO
14	Permuted-MNIST	SGD @ 10^{-3}	eFOPNG EMA	5	**20 tasks** × 5 ep. Fisher = 60000. α = 0.5. Config 414.	TODO

Continued on next page...

Table 2 — *continued from previous page*

#	Benchmark	First task	Methods	Seeds	Notable parameters	Status
15	Permuted-MNIST	SGD @ 10^{-3}	eFOPNG	5	**20 tasks** × 5 ep. Fisher = 60000. $\alpha = 0.5$. Config 415.	TODO
16	Split-MNIST SH HN	AdamW @ 10^{-3}	Adam	1	*Exp 701.* $c = 1000$, $d_h = 8$, emb = 4/10. Sweeps $\beta \in \{0.01, 0.05, 0.1\}$.	✓
17	Split-CIFAR10 HN	AdamW @ 10^{-3}	Adam	1	*Exp 702.* $d_h = 32$, emb = 16/16. Sweeps β and $c \in \{64, 500, 1000, 2000, 5000, 6000\}$.	✓
18	Split-CIFAR100 HN	AdamW @ 10^{-3}	Adam	1	*Exp 703.* **Fast check** (5 tasks × 15 ep). $c = 6000$, $d_h = 128$. Sweeps β.	✓
19	Split-MNIST (SH)	SGD @ sweep lr	SGD	5	**Diagnostic.** Exp 419. 1 task × 5 ep. Sweeps $\eta \in \{10^{-5}, 5{\times}10^{-5}, 10^{-4}, 2{\times}10^{-4}, 5{\times}10^{-4}, 10^{-3}, 5{\times}10^{-3}, 10^{-2}\}$. Fisher = 12000. Batch = 10. **Result:** threshold between $5{\times}10^{-5}$ and 10^{-4}. Paper's FOPNG lr = $10^{-5} \Rightarrow$ SGD acc ≈0.21 (barely above random); protocol is self-contradictory.	✓
20	Split-CIFAR10 (MH)	SGD @ sweep lr	SGD	5	**Diagnostic.** Exp 420. 1 task × 5 ep. Sweeps $\eta \in \{10^{-3}, 5{\times}10^{-3}, 10^{-2}, 5{\times}10^{-2}\}$. Fisher = 1024. Batch = 10. **Result:** paper's $\eta{=}10^{-3}$ yields acc ≈0.85, loss ≈0.65 (above random $\ln 2$). True convergence only at $\eta{\geq}10^{-2}$; refutes paper's first-task protocol for CIFAR-10.	✓

REFERENCES

De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., ... Tuytelaars, T. (2022). A continual learning survey: Defying forgetting in classification tasks. , *44*(7), 3366–3385. Retrieved 2026-05-16, from https://ieeexplore.ieee.org/document/9349197/ doi: 10.1109/TPAMI.2021.3057446

Garg, I., Kolhe, N., Peng, A., & Gopalam, R. (2026). *Fisher-orthogonal projected natural gradient descent for continual learning* (No. arXiv:2601.12816). arXiv. Retrieved from http://arxiv.org/abs/2601.12816 doi: 10.48550/arXiv.2601.12816

Ha, D., Dai, A., & Le, Q. V. (2016). *Hypernetworks* (No. arXiv:1609.09106). arXiv. Retrieved from https://arxiv.org/abs/1609.09106 doi: 10.48550/arXiv.1609.09106

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., ... Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. , *114*(13), 3521–3526. doi: 10.1073/pnas.1611835114

McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. , *24*, 109–165. doi: 10.1016/S0079-7421(08)60536-8

Oswald, J. v., Henning, C., Grewe, B. F., & Sacramento, J. (2022). *Continual learning with hypernetworks* (No. arXiv:1906.00695). arXiv. Retrieved from http://arxiv.org/abs/1906.00695 doi: 10.48550/arXiv.1906.00695

Yadav, Y., Mendoza, P., & Korrapati, J. (2025, Dec). *Ong: Orthogonal natural gradient descent.* Retrieved from https://arxiv.org/abs/2508.17169